

Generator statičkih sajtova

Diplomski rad

Luka Mihailović IT 64/22

Računarski fakultet

Septembar 2025

Mentor: *dr. Marko Mladenović*

Sadržaj

1	Uvod	2
1.1	Istorijski pregled	2
1.2	Problemi i karakteristike postojećih rešenja	4
2	Planiranje i specifikacija sopstvenog rešenja	6
2.1	Izgled direktorijuma <i>sgo</i> projekta i način rada rešenja	6
3	Pisanje sopstvenog rešenja	9
3.1	Raščlanjivanje <i>Markdown</i> datoteka	9
3.2	Prikaz linkova ka stranicama	11
3.2.1	Sortiranje	14
3.3	Demonstracija	15
4	Zaključak	19

1. UVOD

Ovaj rad se bavi konceptom generatora statičkih sajtova, i pružajući uvid u njihovu istoriju, stremi ka tome da ponudi sopstveno rešenje ka problemima koje je autor uočio u trenutnom pejzažu generisanja veb sadržaja.

Istorijski pregled zalazi dublje u okolnosti koje su dovele do potrebe za rešenjem generatora statičkih sajtova, i ujedno ističe jaz koji je prisutan među postojećim rešenjima kada se gleda jednostavnost rada u njima, mogućnosti koje nude, ali i njihov tehnološki dug koje snosi krajnji korisnik koji namerava da ih koristi.

Autor će ponuditi sopstveno rešenje koje cilja da premosti pomenute krajnosti, uočivši nedostatak programa napisanih u ozbiljnim¹ jezicima, a koji su ujedno jednostavni za rad, omogućavajući da se brzo i lako napravi veb-sajt sa što manje učenja toga kako radi sam program, i što manje očekivanja od korisnika.

1.1 Istorijski pregled

Internet i veb-sajtovi, makar kakve ih danas znamo, nisu nastali sve do 1990. godine kada je *Tim Berners-Li* implementirao *HTTP* protokol ([1]). Prikazivanje *hypertext* dokumenata preko novog *HyperText Transfer Protocol*-a ostvareno je pomoću *HyperText Markup Language* tekstualnog formata.

HyperText Markup Language (HTML) i *HTTP* su ostali standard do današnjeg dana.

Rani internet je time karakterisan **statičnim** sajtovima, odnosno sajtovima čiji sadržaj se generalno ne menja, ili menja jako retko. Korisnički zahtevi su vremenom rasli, i ubrzo je bilo očigledno da mora postojati određena interaktivnost i dinamičnost u veb-preglednicima² korisnika.

Prvo rešenje bilo je *CGI (Common Gateway Interface)* programiranje. *CGI* je pružao mogućnost serveru da koristi alternativne programe ili skripte radi obrade *HTTP* zahteva korisnika, pogledati [2].

¹neinterpretirani jezici izvorno namenjeni da se pokreću na serverskoj strani, prim. aut.

²kolokvijalno poznat kao “web pretraživač”, navedeni pojam potiče od prof. D. Pilipovića, prim. aut.

Mogućnosti CGI nisu bile dovoljne za brzorastući broj korisnika sa sve većim zahtevima, i obzirom da je CGI bio veoma neefikasan ([3]), vrlo brzo su počele da se traže ozbiljnije alternative (PHP, ASP, Mod_Perl...).

Iako istorija nije sasvim sigurna oko toga koji generator statičkih sajtova je bio tačno prvi, opšti konsenzus je da je program *HSC* (*HTML Sucks Completely*) najraniji primerak generatora statičkih sajtova kakvim ih danas znamo ([4]). *HSC* sebe nije zvao generatorom, već prosto “*HTML preprocessor*“. Posedovao je elemente koje danas očekujemo, kao što su uslovne naredbe, naredbe uključivanja (*include*), i izlaz programa u vidu obične *HTML* datoteke.

Ideje programa *HSC* su ostale neshvaćene i nezapažene za njegovo vreme, i veb-sajtovi su još neko vreme nastavili da se dinamički generišu za svaki individualni zahtev svakog korisnika.

Tek 2007. godine je napisan prvi pravi moderni generator statičkih sajtova, *Nanoc*, videti [4]. Pisan je u *Ruby* jeziku, doneo je sa sobom podršku za šablone, metapodatke stranica, podršku (za tada relativno nov) *Markdown*³ format, plugin-e...

Naravno, 2008. godine doći će *Jekyll*, koji predstavlja promenu u paradigmi veb-sajtova, i kako gledamo na arhitekturu istih. Služeći kao sinonim za generatore, *Jekyll* postaje orijentir sa kojim se dan-danas porede mnoga rešenja ([5]), i *GitHub*-ov izabrani alat za njihovu hosting platformu, *GitHub Pages*, što sigurno govori dosta.

Dalji razvoj generatora statičkih sajtova će biti određenim delom derivat od *Jekyll*-a, bilo kodom, ili konceptom. Dominantne tehnologije korišćene za pisanje novih alata će biti *JavaScript*, gde će se pritom malo čega originalnog doprineti, pretežno se nudeći rešenje za problem koji je već rešen, ili koji je nepostojeći.

³<https://daringfireball.net/projects/markdown/>

1.2 Problemi i karakteristike postojećih rešenja

U istorijatu su spomenuti konkretni alati bazirani na *Ruby* jeziku, kao i to da je mnoštvo alata pisano u *JavaScript*-u. Vredi spomenuti da je glavna boljka ovih tehnologija to što su pomenuti jezici interpretirani, i što zahtevaju instalaciju kako samog jezika, tako i svojih zavisnosti, na računar krajnjeg korisnika. Alternativa ovome je generator *Hugo*, napisan 2012. godine u *Go* jeziku. Prednost alata, ali i samog jezika, je to što se kompajluje u jednu izvršnu datoteku, koju je dovoljno samo pokrenuti; bez glavobolja oko upravljanja verzijama jezika, paketa, i same instalacije istih.

Hugo predstavlja jedno jako ozbiljno i zrelo rešenje, koje poseduje skoro sve što bi korisniku moglo da zatreba na statičkom sajtu, od taksonomija i tagova na stranicama (objavama), do *RSS* mogućnosti, lokalizacije, i ugrađene manipulacije slikama.

Jedini potencijalni problem (koji je možda stvar ukusa) jeste arhitektura jednog *Hugo* projekta, odnosno sajta. *Hugo* poseduje glavni direktorijum, unutar kojeg se nalazi konfiguraciona datoteka, direktorijum sa *Markdown* datotekama sadržaja, lokalizacijom, itd. Pored ovoga, nalazi se i datoteka sa temama, koja zapravo predstavlja mesto gde se definiše kako će sadržaj da izgleda i da se generiše, sa nekim direktorijumima (npr. *archetypes*, koji predstavljaju programatorski šablon za kreiranje određenih tipova *Markdown* sadržaja) i datotekama (kao što je konfiguraciona) dupliranim, tako da se nalaze na oba mesta.

Ova količina slobode jeste jako moćna, ali ujedno sa sobom i donosi određenu kompleksnost u svaki *Hugo* projekat koji se razvija, i povećava mentalni teret kod ozbiljnih sajtova gde treba paziti koje podešavanje “gazi” preko kojeg, i gde su granice zaduženja temi, a gde samog sajta; da li se tema bavi generisanjem tipova sadržaja, ili sam sajt, koji može imati veći broj tema unutar sebe, ali koje se biraju preko konfiguracione datoteke.

Na kraju svega, glavni problem kod programa *Hugo*, koji je autor uočio, jeste što dolazimo do problema kokoške i jajeta - ne možete imati niti testirati veb-sajt bez već razvijene teme, niti možete razvijati temu bez nekog već postojećeg sadržaja.

Postoje i prostija rešenja pisana u *Go* jeziku, kao što je *staw*, autora Anselma Garbea, glavnog čoveka iza *Suckless* zajednice, videti [6]. Ovo rešenje je jako ograničavajuće,

obzirom da je nastalo uzorom na *werc*⁴ veb-okvir, ciljajući kompatibilnost za migraciju sa istog, umesto da se fokusira na praktične probleme koje bi generator statičkih sajtova trebao da ponudi da reši.

Sa svim ovim na umu, dolazi se do zaključka da ne postoji kompromis na polju generatora statičkih sajtova. Korisnik je primoran da koristi rešenja pisana u sporim, interpretiranim jezicima, ili da se povinuje prema moćnoj i glomaznoj arhitekturi *Hugo*-a (osim ako nije saglasan sa time da bude znatno ograničen idejama već postojećih rešenja). Ovaj rad ima za cilj da premosti taj jaz, i ponudi rešenje koje spaja apsolutnu jednostavnost i minimalizam, zajedno sa čvrstinom i stabilnošću ozbiljnog jezika koji se oslanja na svoju standardnu biblioteku za skoro sve komponente.

Predloženo rešenje time mora i da uzme u obzir da postoji šansa da će manje tehnički sposobna lica želeti da koristi generator statičkih sajtova, što može biti izazovno za ljude koji nisu *developeri*, kao što je opisano u [7]. Stoga, iako je generator statičkih sajtova nasledno *developerski* alat, ne znači da se ne treba potruditi, bilo kroz sâm dizajn rešenja ili dokumentaciju, da rešenje bude što pristupačnije i lakše za rad.

⁴<https://werc.cat-v.org/>

2. PLANIRANJE I SPECIFIKACIJA SOPSTVENOG REŠENJA

Projekat je pisan u *Go* programskom jeziku, a samo rešenje je nazvano *sugo*⁵⁶. Standardna biblioteka jezika *Go* obuhvata skoro sve što bi bilo potrebno na samom projektu, i ujedno nudi mogućnost *template*-inga u sklopu svojih *text/template* i *html/template* paketa, zaobilazeći rešenja trećih stranki, kao što su *Jinja* za *Python* jezik, ili *EJS* za pojedina *JavaScript* rešenja. Ovime se osigurava stabilnost samog programa, i njegova nezavisnost od spoljnih uticaja, znajući da se razvojni tim *Go* jezika strogo pridržava filozofije stabilnosti i funkcionalnosti već postojećeg kôda, tako da isti nema potrebe menjati ili ažurirati u budućnosti zbog novijih verzija samog jezika.

2.1 Izgled direktorijuma *sugo* projekta i način rada rešenja

Autorov projekat je osmišljen da funkcioniše deterministički, sa što manje “pokretnih delova”.

```
-d      run dev server
-i string
      path to website directory (default ".")
-o string
      path for generated static web files (default "website")
```

Listing 1: Rezultat pokretanja komande *sugo -h*

Program se podrazumevano pokreće unutar direktorijuma koji sadrži podatke (sadržaj, šabloni i statične datoteke) kojima planiramo da generišemo veb-sajt (alternativni direktorijum je moguće naglasiti sa argumentom *-i*, listing 1). Ovaj direktorijum ćemo dalje zvati **koren** projekta.

Nakon pokretanja, program će za svaku datoteku unutar *content/* direktorijuma pokušati da nađe odgovarajući šablon unutar *template/* direktorijuma, tako da im je relativna putanja ista. To znači da su relativne putanje identične, ali da je korenski direktorijum različit, gde su korenski direktorijumi za sadržaj i šablon *content/* i *template/*, respektivno.

⁵Od italijanske reči za paradajz sos, najčešće korišćen prilikom pripremanja jela sa testeninom

⁶[GitHub repozitorijum](#)

Rezultat generisanja veb-sajta se podrazumevano smešta u *website/* direktorijum unutar korena projekta (videti argument `-o` unutar listinga 1), tako da je sačuvana struktura podataka navedena u *content/* direktorijumu.

Direktorijumi⁷ koji moraju da se nalaze unutar korena projekta:

content/

sadrži *Markdown* datoteke sa sadržajem veb-sajta

static/

sadrži direktorijume i datoteke koje se “slepo” kopiraju u koren novogenerisanog veb-sajta

templates/

sadrži *.gohtml* datoteke koje služe kao šabloni za sadržaj i podatke iz *Markdown* datoteka unutar *content/* direktorijuma. Mora da ima istu podstrukturu dece direktorijuma kao i *content/*

Tipičan koren projekta (nazvan *lukam.xyz*) za *sugo* izgleda ovako:

```
lukam.xyz
|-- content
|-- static
|   |-- css
|   '-- images
|-- templates
|   '-- _layouts
'-- website
```

Listing 2: Struktura tipičnog *sugo* projekta

Ne treba zaboraviti da je direktorijum *website/* generisan od strane programa, i kao takav ne utiče na rad istog.

Ako imamo strukturu *content/* direktorijuma nalik na listing 3, onda i *template/* direktorijum mora imati istu strukturu poddirektorijuma (listing 4).

⁷*sugo* trenutno ne koristi nikakvu konfiguracionu datoteku, te su jedino potrebni direktorijumi sa podacima


```

content
|-- blog
|   |-- notes-on-using-qemu.md
|   |-- darkmode-difficulties.md
|   |-- index.md
|   '-- lorem.md
|-- hobbies
|   |-- films
|   |   '-- index.md
|   '-- index.md
|-- index.md
'-- software
    |-- index.md
    '-- sugo.md

```

Listing 3: Struktura tipičnog *content/* direktorijuma.

```

templates
|-- blog
|   |-- section.gohtml
|   '-- single.gohtml
|-- hobbies
|   |-- films
|   |   '-- section.gohtml
|   '-- section.gohtml
|-- _layouts
|   |-- base.gohtml
|   |-- footer.gohtml
|   |-- header.gohtml
|   '-- head.gohtml
|-- section.gohtml
'-- software
    |-- section.gohtml
    '-- single.gohtml

```

Listing 4: Struktura *templates/* direktorijuma koja mora da prati *content/* direktorijum.

Svaka *index.md* datoteka predstavlja *index.html* datoteku datog direktorijuma na generisanom veb-sajtu. Kao takva, ona ima jedinstvenu ulogu da pruži uvid u sadržaj

direktorijuma unutar kojeg se nalazi, i time se očekuje da poseduje drugačiji stil od ostalih stranica koje popisuje.

Izabrano ime za šablone *index* stranica je *section.gohtml*, obzirom da omogućavaju pregled sekcije sadržaja.

Svaka datoteka koja nije *index.md* će biti generisana pomoću šablona *single.gohtml*, obzirom da predstavlja jednu stranicu iz date sekcije.

Da bi se izbegla redudancija prilikom pisanja zaglavlja, podnožija, ili bilo kog drugog ponavljajućeg dela šablona, koristi se `__layouts/` direktorijum, koji sadrži osnovnu konstrukciju zajedničku za sve šablone i stranice veb-sajta.

Olakšano testiranje i “brzi” pregled generisanog veb-sajta omogućeno je u lokalu⁸ pokretanjem programa uz argument `-d` (videti listing 1), koji pokreće lokalni server (podrazumevano na portu 8080) koristeći datoteke iz direktorijuma sa generisanim veb-sajtom.

3. PISANJE SOPSTVENOG REŠENJA

3.1 Raščlanjivanje *Markdown* datoteka

Ranije u radu je ustanovljeno da je veliko ograničenje unapred odrediti korisniku koje meta-podatke je moguće proslediti svakoj stranici. Ti meta-podaci su kolokvijalno poznati kao *front matter*, obzirom da se često pišu na početku *Markdown* datoteka sadržaja, odvojeni nekim učestalim karakteristikama, tako da je i korisniku i programu lako da razgraniče sam sadržaj od *front matter* meta-podataka.

Autorov program omogućava zapisivanje *front matter* podataka isključivo u *JSON* formatu, uprkos većini generatora statičkih sajtova, koji dozvoljavaju više različitih formata zapisa podataka (npr. [8] i [9]), razlikovanih pomoću niza karaktera za odvajanje *front matter* podataka od samog sadržaja ([8]). Razlog iza ove odluke je već postojeća biblioteka za čitanje *JSON* zapisa u *Go* jeziku, i odsustvo podrške za ostale česte formate unutar standardne biblioteke, kao što su *yaml*, i njegov prostiji i jednostavniji srodnik, *toml*. Obzirom da se autorov program trudi da ne zavisi od biblioteka pisanih od strane drugih

⁸računar koji koristi sâm korisnik

stranki, ovo ograničenje je očekivano.

```
1 // GetFrontMatter extracts the json formatted front matter from a
   content file. Returns the front matter of said file and the index at
   which Markdown content starts.
2 func GetFrontMatter(filePath string, delimiter string) (map[string]any,
   int, error) {
3     fileBytes, err := os.ReadFile(filePath)
4     if err != nil {
5         return nil, -1, err
6     }
7
8     fileContent := string(fileBytes)
9
10    startIndex := strings.Index(fileContent, delimiter)
11    if startIndex == -1 {
12        return nil, -1, errors.New("no front matter delimiter found in
           content file")
13    }
14    startIndex += len(delimiter)
15
16    endIndex := strings.Index(fileContent[startIndex:], delimiter)
17    endIndex += len(delimiter)
18
19    frontMatter := "{" + fileContent[startIndex:endIndex] + "}\n"
20
21    if !json.Valid([]byte(frontMatter)) {
22        return nil, -1, errors.New("invalid json in front matter: " +
           frontMatter)
23    }
24    data := make(map[string]any, 0)
25    err = json.Unmarshal([]byte(frontMatter), &data)
26    if err != nil {
27        return nil, endIndex + len(delimiter), err
28    }
29    return data, endIndex + len(delimiter), nil
30 }
```

Listing 5: Funkcija za izvlačenje *front matter* meta-podataka sa početka *Markdown* sadržajnih datoteka.

Jedina tačka gde se autorov program oslanja na strane biblioteke je samo raščlanjivanje *Markdown* sadržaja, za koje se koristi *goldmark* paket [10], odnosno biblioteka, koja se koristi na linijama 11 i 13 funkcije prikazanoj unutar listinga broj 6.

```
1 // GetTextContent reads a Markdown file, processes its content from a
   given offset, and converts it to HTML text.
2 func GetTextContent(filePath string, offset int) (string, error) {
3     fileBytes, err := os.ReadFile(filePath)
4     if err != nil {
5         return "", err
6     }
7
8     fileContent := fileBytes[offset:]
9     var buf bytes.Buffer
10
11     mdParser := goldmark.New(goldmark.WithExtensions(extension.GFM))
12
13     err = mdParser.Convert(fileContent, &buf)
14     if err != nil {
15         return "", err
16     }
17
18     return buf.String(), nil
19 }
```

Listing 6: Funkcija za izvlačenje samog tekstualnog sadržaja iz *Markdown* datoteke.

3.2 Prikaz linkova ka stranicama

Obzirom da je za mnoge slučajeve korišćenja neophodno imati linkove koji vode ka ostalim stranicama neke sekcije, ili generalno navigacione menije, ubrzo se uočila potreba za funkcijom koju korisnik može pozvati iz šablonske datoteke, a koja bi kao rezultat izvršavanja vraćala neku strukturu podataka sa podacima o svim stranicama prisutnim unutar nekog direktorijuma.

Oslanjanje na hijerarhiju *content/* direktorijuma za konačan izgled direktorijuma generisanog veb-sajta omogućilo je logički jednostavnu implementaciju ove funkcionalnosti, prikazano u Listingu 7.

```

1 // GetChildPages scans the specified content directory for markdown
  files and returns a map of their HTML file names mapped to their
  titles.
2 func GetChildPages(url string, indexesOnly bool) map[string]map[string]
  any {
3   var pages = make(map[string]map[string]any)
4   root := filepath.Join("content", url)
5   rootDepth := strings.Count(root, string(os.PathSeparator))
6
7   err := filepath.WalkDir(root, func(path string, d fs.DirEntry, err
  error) error {
8     if filepath.Ext(path) == ".md" {
9       htmlFilePath := strings.TrimSuffix(path, filepath.Ext(path))
10      htmlFilePath += ".html"
11
12      if !indexesOnly && strings.Contains(htmlFilePath, "index.html") {
13        return nil
14      } else if indexesOnly && !strings.Contains(htmlFilePath, "index.
  html") {
15        return nil
16      }
17
18      relPath, err := filepath.Rel("content", htmlFilePath)
19      if err != nil {
20        log.Fatal(err)
21      }
22
23      relDepth := strings.Count(relPath, string(os.PathSeparator))
24
25      if d.IsDir() && (indexesOnly && relDepth-rootDepth == 0) ||
  relDepth-rootDepth > 1 {
26        return filepath.SkipDir
27      }
28
29      if indexesOnly {
30        relDepth := strings.Count(relPath, string(os.PathSeparator))
31        if relDepth-rootDepth == 0 {
32          return nil

```

```

33     }
34 }
35
36     fullPath := filepath.Join("/", relPath)
37
38     pageData, _, err := GetFrontMatter(path, "+++")
39     if err != nil {
40         log.Fatal(err)
41     }
42
43     pages[fullPath] = pageData
44 }
45 return nil
46 })
47 if err != nil {
48     log.Fatal(err)
49 }
50 return pages
51 }

```

Listing 7: Funkcija za dohvaćanje stranica jedan nivo ispod zadatog direktorijuma, dohvatajući indeksne ili sadržajne datoteke, zavisno od prosleđenog argumenta

Primer korišćenja ove funkcije radi generisanja navigacionog menija unutar šablona zaglavlja je prikazan na listingu 8. Primetiti kako se na liniji 2 ručno navodi link sa putanjom ka početnoj strani veb-sajta. Ovo je ograničenje funkcije za nabavljanje linkova ka stranicama iz listinga 7, obzirom da funkcija ne vraća sadržaj iz direktorijuma koji joj je prosleđen kao argument, nemoguće je dobiti putanju za datoteke prisutne u korenu veb-sajta, odnosno u korenu *content/*.

Način prikazivanja podataka unutar šablona regulisan je pomoću *text/template* paketa za *Go*. Konsultovati se sa zvaničnom dokumentacijom iz [11].

```

1 <nav>
2     <a href="/">Home</a>
3     {{ range $link, $data := GetChildPages "" true }}
4         <a href="{{ $link }}"> {{ $data.Title }} </a>
5     {{ end }}
6 </nav>

```

Listing 8: Primer korišćenja funkcije iz listinga 7 unutar neke šablon datoteke.

Sortiranje

Po specifikaciji implementacije strukture podataka mapa jezika *Go*, vršenje iteracije kroz elemente mape je nedeterministička (stohastička) radnja [12]. Pošto je čest slučaj upotrebe za veb-sajtove prikazivanje nekih članaka ili sadržaja sortirano (bilo datumski ili leksikografski), bilo je potrebno napisati dodatnu funkciju koja bi rezultat prethodno navedene funkcije za nabavljanje linkova stranica (Listing 7) sortirala po korisničkom kriterijumu.

```

1 // SortPages sorts a map of pages by the specified key in ascending or
   // descending order, returning a slice of pages.
2 func SortPages(pages map[string]map[string]any, sortKey string, desc
   bool) (sortedPages []map[string]any) {
3     out := make([]map[string]any, 0)
4     for link, data := range pages {
5         data["Link"] = link
6         out = append(out, data)
7     }
8
9     if sortKey == "Date" {
10        sort.Slice(out, func(i, j int) bool {
11            t1, err := time.Parse("2-1-2006", out[i][sortKey].(string))
12            if err != nil {
13                log.Fatal(err)
14            }
15
16            t2, err := time.Parse("2-1-2006", out[j][sortKey].(string))
17            if err != nil {

```

```

18         log.Fatal(err)
19     }
20
21     if desc {
22         return t1.After(t2)
23     } else {
24         return t1.Before(t2)
25     }
26 })
27
28     return out
29 }
30
31 sort.Slice(out, func(i, j int) bool {
32     if desc {
33         return out[i][sortKey].(string) > out[j][sortKey].(string)
34     }
35     return out[i][sortKey].(string) < out[j][sortKey].(string)
36 })
37
38     return out
39 }

```

Listing 9: Funkcija za sortiranje mape stranica, formata *link: podaci*.

Korišćena je struktura *slice* (tip koji apstrahuje rad sa nizovima), pošto se ona smatra najboljom praksom za predstavljanje sortiranih vrednosti, videti [12] i način implementacije sortiranja vrednosti mape.

3.3 Demonstracija

Trenutno stanje direktorijuma sa sadržajem, tj. *Markdown* datotekama je prikazano na listingu 3. Za taj sadržaj imamo već definisane šablone, prikazano na listingu 4.


```
static
|-- css
|   '-- style.css
'-- images
    '-- pf2_2fort_logo.jpg
```

Listing 10: Sadržaj *static/* datoteke

Istaknućemo i sadržaj direktorijuma *static/*, gde čuvamo naš *CSS* i sliku koju koristimo u datoteci na putanji *content/blog/smith.md*, njen sadržaj prikazan je na listingu 10.

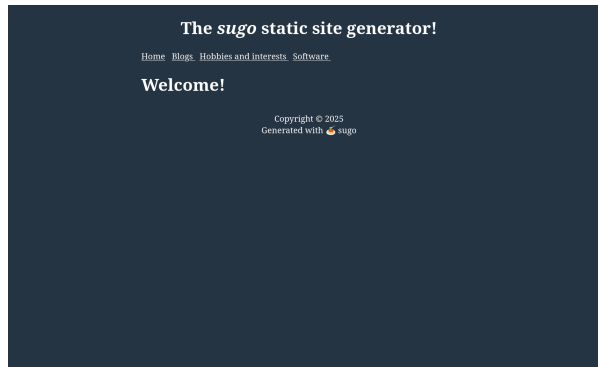
Kada smo spremni, pokrenućemo komandu `./sugo`⁹. Primetićemo novonastali direktorijum *website/*, koji sadrži sve datoteke našeg generisanog veb-sajta (slika 1). Sadržaj direktorijuma *website/* spreman je da se prekopira u direktorijum sa javnim *HTML*-om unutar veb-servera.

```
$ tree website
website
├── blog
│   ├── customisation.html
│   ├── darkmode-difficulties.html
│   ├── index.html
│   └── smith.html
├── css
│   └── style.css
├── hobbies
│   └── films
│       └── index.html
├── images
│   └── pf2_2fort_logo.jpg
├── index.html
├── software
│   ├── index.html
│   └── sugo.html
```

Slika 1: Izgled *website/* direktorijuma nakon pokretanja *sugo*

Ukoliko želimo da na lokalnoj mašini proverimo kako izgleda naš sajt, dovoljno je pokrenuti `./sugo -d`, i posetiti `http://localhost:8080`.

⁹sintaksa *UNIX* sistema. Pokretanje na platformi *Windows* se ostavlja kao vežba čitaocu.



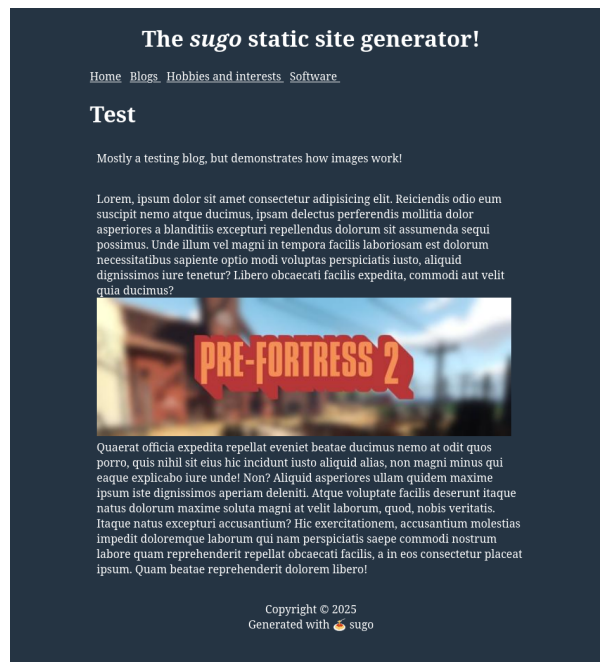
Slika 2: Izgled *home* stranice generisanog veb-sajta, skalirano na 150% prvobitne veličine.

Na slikama 2, 3 i 4, pri vrhu stranice, vidi se navigacioni meni sa stavkama *Home*, *Blog*, *Hobbies and interests* i *Software*. Na Slici 3 vidi se rezultat funkcije listing 7, koji je prošao kroz funkciju za sortiranje po datumu, prikazana u listingu 9.



Slika 3: Izgled generisane stranice *blog*, sa svim objavama vidljivim i sortiranim po datumu, od novijih ka starijim. Skalirano na 150% prvobitne veličine.

Na Slici 4 vidi se rezultat generisanja *Markdown* datoteke koja u sebi sadrži sliku, i ujedno ima svojstvo *Description*, koje je unutar šablona *template/blog/single.gohtml* definisano da se prikazuje odmah ispod naslova ukoliko postoji.



Slika 4: Izgled generisane blog objave. Pasus odmah ispod naslova *Test* prikazuje vrednost svojstva *Description* iz *front matter* dela datoteke sa sadržajem.

4. ZAKLJUČAK

Cilj autorovog rešenja jeste alat koji bi najpre služio njima, napisan sa vrednostima minimalizma i korektnog kôda, ali i jednostavnosti korišćenja. Autorovo rešenje time predstavlja povratak korenima na neki način: nekomplikovano činjenje jedne delatnosti za koju je osmišljen, i ništa preko.

Neke funkcionalnosti trenutno nisu implementirane, kao što su tagovi, odnosno ključne reči stranica, procesovanje fotografskih datoteka, multilingvalnost, minimizovanje *CSS* i *JavaScript* datoteka, ili paginacija (o *plugin* sistemu se može samo maštati). Sve ovo se smatra nečim što je ili van domena programa, ili za čime još nema dovoljno velike potrebe da opravda dodatnu kompleksnost koja bi se unela u izvorni kôd programa.

Sugo je nastao iz frustracije i nezadovoljstvom sa postojećim rešenjima, i garantovano je da će *sugo* zauzvrat frustrirati i učiniti druge nezadovoljnim. To je program sa veoma čvrstim mišljenjem i stavovima, nameračen da radi samo jednu stvar, i ništa preko; a to je generisanje statičkih sajtova¹⁰.

¹⁰Za primer rada *sugo* programa, posetiti autorov veb-sajt, <https://www.lukam.xyz>

BIBLIOGRAFIJA

1. Mozilla. *Evolution of HTTP* [Online; accessed 9-september-2025]. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Evolution_of_HTTP.
2. Wikipedia. *Common Gateway Interface* [Online; accessed 9-september-2025]. https://en.wikipedia.org/wiki/Common_Gateway_Interface.
3. Labrinidis, A. & Roussopoulos, N. Generating dynamic content at database-backed web servers: cgi-bin vs. mod_perl. *ACM SIGMOD Record* **29**, 26–31 (2000).
4. Neumegen, M. *The 'Before Jekyll' era* [Online; accessed 9-september-2025]. <https://cloudcannon.com/blog/ssg-history-1-before-jekyll/>.
5. Neumegen, M. *The 'After Jekyll' era* [Online; accessed 9-september-2025]. <https://cloudcannon.com/blog/ssg-history-2-after-jekyll/>.
6. Garbe, A. *staw* [Online; accessed 12-september-2025]. <https://github.com/garbeam/staw>.
7. Petersen, H. From static and dynamic websites to static site generators. *University of TARTU, Institute of Computer Science* (2016).
8. *Hugo Front Matter* [Online; accessed 12-september-2025]. <https://gohugo.io/content-management/front-matter/>.
9. *Jekyll Front Matter* [Online; accessed 12-september-2025]. <https://jekyllrb.com/docs/front-matter/>.
10. Inuzuka, Y. *A markdown parser written in Go* [Online; accessed 12-september-2025]. <https://github.com/yuin/goldmark>.
11. The Go Team. *text/template* [Online; accessed 13-september-2025]. <https://pkg.go.dev/text/template>.
12. Gerrand, A. *Go maps in action* [Online; accessed 14-september-2025]. 2013. <https://go.dev/blog/maps>.